ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN KẾT THÚC HỌC PHẦN THUẬT TOÁN ỨNG DỤNG

Đề tài: "TÌM ĐƯỜNG NGẮN NHẤT BẰNG THUẬT TOÁN DIJKSTRA TRÊN BẢN ĐỒ HÀ ĐÔNG"

Sinh viên : Lê Đức Thiện

LỚP : Thuật toán ứng dụng

Năm học : 2025-2026

Giảng viên hướng dẫn : Trần Đình Tân

HÀ NỘI, THÁNG 11/2026

BẢNG PHÂN CÔNG NHIỆM VỤ

Họ và tên	Mã sinh viên	Nhiệm vụ	Hoành thành
Lê Đức Thiện	23010186	-Lấy dữ liệu -Áp dụng thuật toán -Phân tích bài toán -Xử lí bài toán -Tổng kết	100%

MỤC LỤC

Chương 1. Giới thiệu đề tài 3
1.1. Lý do chọn đề tài 3
1.2. Mục tiêu đề tài 3
1.3. Phạm vi nghiên cứu 3
Chương 2. Dữ liệu bản đồ 4
2.1. Nguồn dữ liệu 4
2.2. Dạng dữ liệu 4
2.3. Trọng số dùng trong thuật toán 4
2. 4. File dữ liệu 4
2.5. Quy mô dữ liệu 5
2.6. Dữ liệu mẫu 5
Chương 3. CƠ SỞ LÝ THUYẾT 6
3.1. Khái niệm đồ thị 6
3.2. Dữ liệu bản đồ OpenStreetMap (OSM) 6
3. 3. Thuật toán Dijkstra 7
3.4 Thuật toán A* 8
Chương 4. THIẾT KẾ HỆ THỐNG 10
4.1. Mục tiêu thiết kế hệ thống 10
4.2. Kiến trúc tổng thể của hệ thống 11
4.3. Cấu trúc thư mục hệ thống 12
4.4. Luồng hoạt động của hệ thống 13
4.5. Thiết kế chức năng chính 14
5.GIAO DIỆN VÀ KẾT QUẢ THỰC NGHIỆM 17
5.1. Mục đích thực nghiệm 17
5.2. Giao diện hệ thống 17
Chương 6. ĐÁNH GIÁ 20
6.1. So sánh thuật toán Dijkstra với A* 20

6.2 Đánh giá về độ chính xác của thuật toán 22
6.3. Những xu hướng quan trọng rút ra từ thực nghiệm 22
6.4. Ưu điểm của hệ thống 22
6.5. Hạn chế 23
6.6. Kết luận chương 23
Chương 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 23
7.1. Kết luận 23
7.2. Những điểm mạnh của đề tài 24
7.3. Hạn chế 25
7.4. Hướng phát triển 25
7.5. Kết luận chung 26

Chương 1. Giới thiệu đề tài

1.1. Lý do chọn đề tài

Trong bối cảnh đô thị hóa nhanh tại Hà Nội, nhu cầu tìm các tuyến đường ngắn nhất để di chuyển hiệu quả đang ngày càng trở nên quan trọng. Các ứng dụng bản đồ như Google Maps, Here Maps hay các hệ thống điều hướng giao thông thông minh đều dựa trên những thuật toán tìm đường tối ưu để hỗ trợ người dùng.

Quận Hà Đông là một khu vực đông dân cư, mật độ giao thông cao, nhiều nút giao và tuyến đường phức tạp. Việc nghiên cứu và xây dựng một hệ thống có khả năng tìm đường đi ngắn nhất trên bản đồ Hà Đông không chỉ giúp hiểu rõ cách vận hành của các thuật toán định tuyến mà còn có ý nghĩa thực tiễn trong tối ưu hóa giao thông và hỗ trợ người dùng lựa chọn lộ trình hiệu quả hơn.

Trong số các thuật toán tìm đường, Dijkstra là một trong những thuật toán kinh điển, mạnh mẽ và được ứng dụng rộng rãi trong nhiều lĩnh vực, đặc biệt là trong tìm đường trên bản đồ. Việc áp dụng thuật toán Dijkstra trên dữ liệu bản đồ thực tế (OpenStreetMap) sẽ giúp sinh viên hiểu sâu hơn cách thuật toán hoạt động trong môi trường thực tế, thay vì chỉ trên đồ thị lý thuyết.

1.2. Mục tiêu đề tài

Đề tài hướng đến việc xây dựng một hệ thống đơn giản nhưng hoàn chỉnh, có khả năng:

- Chuyển đổi dữ liệu bản đồ OpenStreetMap khu vực Hà Đông thành một đồ thị có trọng số.
- Cài đặt thuật toán Dijkstra để tìm đường đi ngắn nhất giữa hai điểm người dùng nhập vào.
- Hiển thị kết quả trực quan trên bản đồ, bao gồm tuyến đường tối ưu, chiều dài tuyến và số nút được mở rộng.
- Đánh giá tốc độ xử lý, hiệu quả và độ hợp lý của tuyến đường tìm được.

1.3. Phạm vi nghiên cứu

- Khu vực nghiên cứu: Quận Hà Đông, Hà Nội.

-Dữ liệu sử dụng: Bản đồ giao thông đường bộ từ OpenStreetMap (OSM).

-Thuật toán áp dụng: Thuật toán Dijkstra với trọng số cạnh là chiều dài đường (meters).

- Hệ thống hiển thị: Web app đơn giản sử dụng Flask + Folium để hiển thị bản đồ.

- Không bao gồm:

- Dữ liệu giao thông thời gian thực (kẹt xe, cấm đường).
- Các loại phương tiện (ô tô, xe máy, đi bộ).
- Tối ưu hóa đa mục tiêu (thời gian, chi phí, nhiên liệu).
- Các thuật toán nâng cao như A*, Yen-k Shortest Paths,...

Chương 2. Dữ liệu bản đồ

2.1. Nguồn dữ liệu

Dữ liệu được lấy từ OpenStreetMap (OSM) thông qua thư viện OSMnx.

Phạm vi tải dữ liệu:

```
# --- Định nghĩa trực tiếp config ---
PLACE = "Ha Dong District, Hanoi, Vietnam"
NETWORK_TYPE = "all_private" #Số nút: 11155, số cạnh: 28044
DATA_DIR = os.path.join(os.path.dirname(__file__), "data")
MAP_FILE = os.path.join(DATA_DIR, "hadong_drive.graphml")
```

Phạm vi: Ha Dong

2.2. Dạng dữ liệu

- Dữ liệu được tải về dạng đồ thị hướng (MultiDiGraph) với các thông tin:
    - Node: tọa độ (lat, lon), độ cao, số đường giao nhau (street_count), v.v.
    - Edge: chiều dài (length), loại đường (highway), hướng đi (oneway)

2.3. Trọng số dùng trong thuật toán

- Trọng số của mỗi cạnh được chọn là:

Chiều dài cạnh (meters) – thuộc tính length của OSMnx.

2. 4. File dữ liệu

- Dữ liệu sau khi tải được lưu thành file:

  hadong_drive.graphml

- File .graphml giúp:
    - Không phải tải lại OSM mỗi lần chạy → tăng tốc.
    - Làm dữ liệu cố định để đánh giá ổn định.

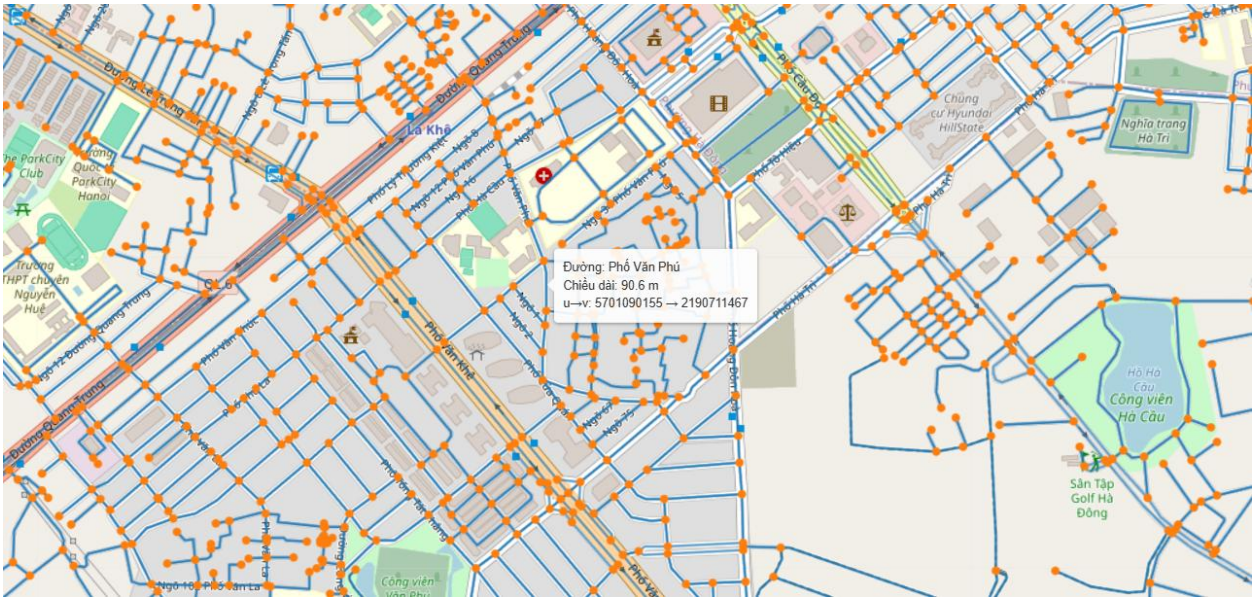
2.5. Quy mô dữ liệu

Dữ liệu Hà Đông (network_type="drive") có:

Chỉ số	Giá trị
Số node	11155
Số cạnh	28.000
Loại đồ thị	MultiDiGraph

Đây là quy mô trung bình phù hợp để thử nghiệm thuật toán Dijkstra,A*.

2.6. Dữ liệu mẫu



Dữ liệu chứa:

Các điểm nút (nodes)

Các cạnh (edges)

Chương 3. CƠ SỞ LÝ THUYẾT

3.1. Khái niệm đồ thị

3. 1.1. Định nghĩa

Đồ thị (Graph) là cấu trúc dữ liệu dùng để mô tả mối quan hệ giữa các đối tượng.
Một đồ thị $G = (V, E)$gồm:

- V: tập đỉnh (nodes/vertices)
- E: tập cạnh (edges) nối giữa các đỉnh

3.1.2. Phân loại đồ thị

- Đồ thị vô hướng (Undirected Graph)
  Cạnh không có hướng, di chuyển hai chiều.
- Đồ thị có hướng (Directed Graph)
  Cạnh có hướng đi từ đỉnh này sang đỉnh khác.
- Đồ thị có trọng số (Weighted Graph)
  Mỗi cạnh có giá trị (chiều dài, chi phí, thời gian…).
- Đồ thị đa cạnh (MultiGraph)
  Có thể tồn tại nhiều cạnh song song giữa hai đỉnh (trường hợp OSMnx).

3.1.3. Biểu diễn đồ thị

- Ma trận kề (Adjacency Matrix) – dùng cho đồ thị nhỏ.
- Danh sách kề (Adjacency List) – phù hợp với đồ thị lớn.

Trong đề tài này, đồ thị được biểu diễn bằng NetworkX dưới dạng:

MultiDiGraph – đồ thị có hướng, cho phép nhiều cạnh giữa hai node.

3.2. Dữ liệu bản đồ OpenStreetMap (OSM)

3. 2.1. Tổng quan về OSM

OpenStreetMap là cơ sở dữ liệu bản đồ mở, miễn phí, được đóng góp bởi cộng đồng.
OSM cung cấp các dữ liệu:

- tọa độ đường (nodes),

- thông tin đường phố (edges),
- loại đường (highway),
- chiều dài (length),
- hướng (oneway),
- tốc độ (maxspeed) *nếu có*.

3.2.2. Thư viện OSMnx

OSMnx cho phép:

- tải dữ liệu OSM vào Python,
- chuyển thành đồ thị trọng số,
- phân tích mạng đường,
- lưu/đọc dưới dạng .graphml.

3. 3. Thuật toán Dijkstra

3. 3.1. Giới thiệu

Thuật toán Dijkstra được đề xuất bởi Edsger Dijkstra (1956), dùng để tìm đường đi ngắn nhất từ một đỉnh nguồn đến tất cả các đỉnh khác trong đồ thị trọng số dương.

Đây là thuật toán cốt lõi trong:

- Google Maps, GPS
- hệ thống điều hướng
- mạng máy tính

3.3.2 Cơ chế hoạt động

Khởi tạo:

```
# Khởi tạo
distances = {node: float('inf') for node in G.nodes()} #lưu khoảng cách ngắn nhất hiện tại đến mỗi node.
distances[start_node] = 0
previous = {node: None for node in G.nodes()} #lưu node trước đó trên đường đi ngắn nhất.
visited = set() #tập các node đã được xử lý.

# Priority queue: (distance, node)
pq = [(0, start_node)]
```

- distances: lưu khoảng cách ngắn nhất hiện tại.

- previous: dùng để truy vết đường đi sau cùng.

- visited: đánh dấu node đã duyệt xong.

- pq: hàng đợi ưu tiên (min-heap) giúp chọn node gần nhất.

Chọn node gần nhất:

```
current_dist, current_node = heapq.heappop(pq) #lấy node có khoảng cách nhỏ nhất hiện tại.
```

Lấy node có khoảng cách nhỏ nhất từ heap.

Đây là tối ưu quan trọng giúp thuật toán chạy nhanh (O(log V)).

Dừng khi đến đích:

```
if current_node == end_node:
    break
```

Relax cạnh:

```
new_dist = current_dist + weight

if new_dist < distances[neighbor]: #Nếu tổng khoảng cách mới new_dist nhỏ hơn khoảng cách cũ → cập nhật:
    distances[neighbor] = new_dist
    previous[neighbor] = current_node
    heapq.heappush(pq, (new_dist, neighbor))
```

Kết quả:

- Thuật toán trả về:
    - đường đi (nodes)
    - chiều dài (length_m, length_km)
    - số node duyệt (nodes_explored)
    - thời gian thực thi (time_ms)

Điều này cho phép đánh giá hiệu năng một cách chính xác trên bản đồ thực tế.

3.4 Thuật toán A*

3.4.1. Mục đích

- Tăng tốc tìm đường bằng cách hướng thuật toán về phía đích.

- Dùng heuristic Haversine dự đoán khoảng cách thẳng từ node hiện tại → node đích.

3.4.2 Heuristic sử dụng

```python
def haversine_distance(lat1, lon1, lat2, lon2):
    """Tính khoảng cách Haversine giữa 2 điểm (mét)"""
    R = 6371000  # Bán kính Trái Đất (mét)

    lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = sin(dlat/2)**2 + cos(lat1) * cos(lat2) * sin(dlon/2)**2
    c = 2 * atan2(sqrt(a), sqrt(1-a))

    return R * c
```

- Tính khoảng cách "đường chim bay" giữa hai điểm theo tọa độ lat/lon.
- Đơn vị: mét
- Đây là heuristic admissible ⇒ đảm bảo A* luôn tìm đường tối ưu.

3.4.3 Cơ chế hoạt động

Khởi tạo:

```
g_scores = {node: float('inf')}
g_scores[start_node] = 0
```

mọi node có khoảng cách vô cùng, riêng node đầu = 0.

Tính f(n) cho node bắt đầu:

```
h_start = haversine_distance(start_lat, start_lon, end_lat, end_lon)
pq = [(h_start, start_node)]
```

Node đầu tiên có f(n) = g(n) + h(n) = 0 + (khoảng cách thẳng đến đích).

Lấy node có f nhỏ nhất:

```
_, current_node = heapq.heappop(pq)
```

→ Đây là tối ưu quan trọng:
Không phải node nào gần điểm xuất phát nhất, mà là node có tiềm năng đi đến đích nhanh nhất.

Relax cạnh giống Dijkstra:

```
new_g = current_g + weight
if new_g < g_scores[neighbor]:
    g_scores[neighbor] = new_g
    previous[neighbor] = current_node
```

Tính lại f(n) cho mỗi neighbor:

```
h = haversine_distance(neighbor_lat, neighbor_lon, end_lat, end_lon)
f_score = new_g + h
heapq.heappush(pq, (f_score, neighbor))
```

Điểm khác biệt quan trọng nhất với Dijkstra:

- Không chỉ xét chi phí đã đi (g)
- Mà còn xét chi phí dự đoán còn lại (h)

→ Node nào tiềm năng đến đích nhanh nhất sẽ được ưu tiên xét trước.

Chương 4. THIẾT KẾ HỆ THỐNG

4.1. Mục tiêu thiết kế hệ thống

Hệ thống được xây dựng nhằm hỗ trợ người dùng nhập vào hai vị trí bất kỳ trên bản đồ quận Hà Đông, sau đó tự động:

1. Chuyển địa chỉ hoặc tọa độ thành node của đồ thị OSM.
2. Áp dụng các thuật toán tìm đường (Dijkstra và A*) để tính tuyến đường tối ưu.
3. Trả kết quả trực quan dưới dạng bản đồ và thông tin tuyến đường.

Do đó, hệ thống phải đảm bảo:

- Hoạt động chính xác với dữ liệu bản đồ thực tế.
- Tốc độ xử lý nhanh nhờ tối ưu thuật toán.

- Giao diện thân thiện, dễ sử dụng.
- Cấu trúc mã rõ ràng, tách biệt chức năng.

4.2. Kiến trúc tổng thể của hệ thống

Hệ thống được xây dựng theo mô hình 3 lớp:

(1) Lớp giao diện (Frontend)

- HTML giao diện nhập điểm bắt đầu/kết thúc.
- Kết quả hiển thị bằng bản đồ Folium (Leaflet).

(2) Lớp xử lý nghiệp vụ (Backend – Flask)

- Nhận input người dùng.
- Xử lý địa chỉ, chuyển thành node gần nhất.
- Gọi thuật toán tìm đường (Dijkstra, A*).
- Trả kết quả về giao diện.

(3) Lớp dữ liệu

- Dữ liệu bản đồ từ OpenStreetMap.
- File hadong.graphml (đã chuẩn hóa từ prepare_graph.py).
- Cache giúp tăng tốc tải đồ thị.

Sơ đồ kiến trúc:

[User Web]

↓

[Flask Controllers]

↓

[Routing Services (Dijkstra/A*)]

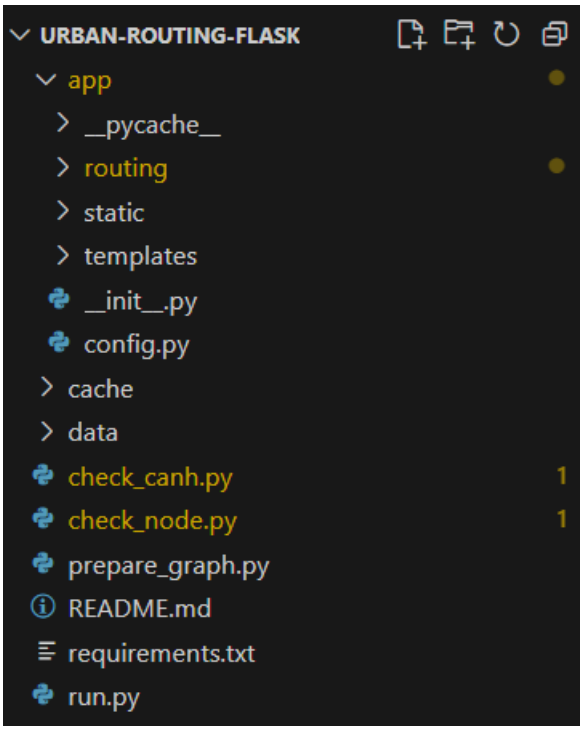
↓

[Graph Data (.graphml)]

↓

[Map Visualization (Folium)]

4.3. Cấu trúc thư mục hệ thống



Ý nghĩa từng thành phần chính:

4.3.1. /app/__init__.py

- Khởi tạo Flask App.
- Nạp cấu hình từ config.py.
- Đăng ký Blueprint routing.

4.3.2. /app/config.py

Chứa:

- Phạm vi bản đồ (Ha Dong, Hanoi).
- Đường dẫn file graphml.
- Cấu hình zoom map.

4.3.3. /app/routing/

Thành phần quan trọng nhất:

File	Chức năng
controllers.py	Nhận dữ liệu từ người dùng, điều phối tìm đường
services.py	Chứa thuật toán Dijkstra & A* (code bạn đã viết)
utils.py	Hàm phụ (geocode, nearest node, parse tọa độ)
visualizer.py	Vẽ tuyến đường trên bản đồ Folium
graph_loader.py	Tải, cache đồ thị OSMnx

4.4. Luồng hoạt động của hệ thống

Bước 1 – Người dùng nhập dữ liệu

Nhập:

- Địa chỉ
- Hoặc tọa độ dạng "lat, lon"

HTML gửi request POST đến Flask.

Bước 2 – Xử lý input

File utils.py:

- Kiểm tra chuỗi có phải tọa độ (regex)
- Nếu không → geocode qua OSMnx
- Xác định node gần nhất bằng:

Bước 3 – Tìm đường

- File services.py gọi:

  Trả về:

- Danh sách node trên đường đi
- Chiều dài tuyến
- Số node duyệt

- Thời gian chạy

Bước 4 – Trực quan hóa

File visualizer.py dùng Folium:

- Vẽ bản đồ Hà Đông
- Vẽ polyline tuyến đường
- Tô màu đỏ đường đi tối ưu

Kết quả render ra route_result.html.

Bước 5 – Hiển thị kết quả

Giao diện cho thấy:

- Tuyến đường (màu cam)
- Chiều dài (m, km)
- Thuật toán nhanh nhất
- Thời gian xử lý
- Số node duyệt

4.5. Thiết kế chức năng chính

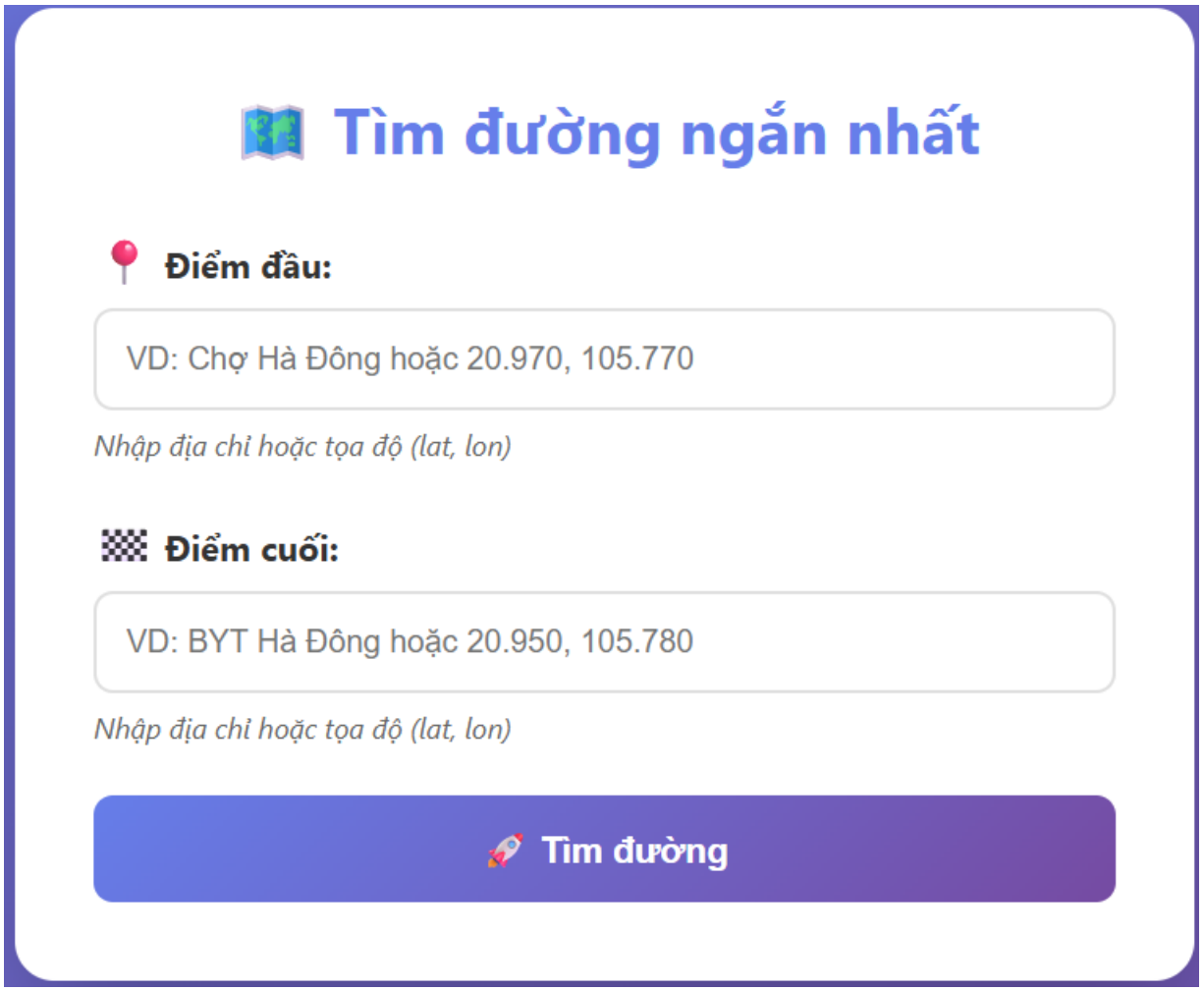
4.5.1. Chức năng nhập vị trí

Người dùng nhập:

- Tên đường / địa chỉ
- Hoặc tọa độ

Hệ thống hỗ trợ cả hai.

Demo:



4..5.2. Chức năng tìm đường

Hệ thống chạy cả 2 thuật toán:

- Dijkstra
- A*

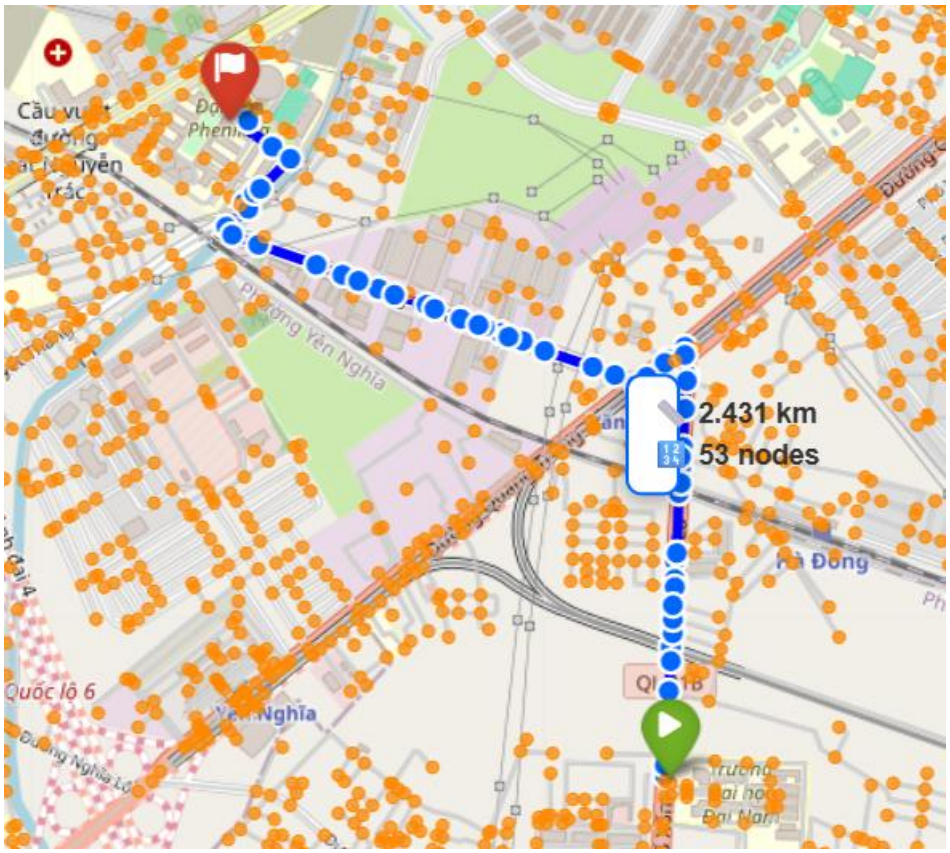
```
#  DÙNG DIJKSTRA TỰ THIẾT KẾ
print(" Đang chạy thuật toán Dijkstra...")
result = dijkstra_shortest_path(_G, start_node, end_node)

# #  DÙNG A* TỰ THIẾT KẾ
# print(" Đang chạy thuật toán A*...")
# result = astar_shortest_path(_G, start_node, end_node)
```

-Em vẫn để chỉnh thuật toán ở trong code ạ

4. 5.3. Chức năng hiển thị bản đồ

- Map nền: Leaflet
- Dùng Folium tạo HTML
- Hiển thị đường đi đỏ
- Zoom tự động theo bounding box đường đi



4 5.4. Chức năng kiểm tra dữ liệu

Hai file:

- check_node.py
- check_canh.py

Giúp kiểm tra:

- Tọa độ node
- Số cạnh
- Độ dài đường

→ Đảm bảo dữ liệu graph đúng trước khi dùng thuật toán.

4.5.5.Nhận Xét:

- Rõ ràng, tách biệt chức năng

- Dễ mở rộng thêm thuật toán (BFS, Yen-k shortest paths…)

- Tối ưu tốc độ nhờ cache và file graphml

- Giao diện đơn giản, dễ dùng

- Cấu trúc chuẩn mô hình MVC mini cho Flask

5.GIAO DIỆN VÀ KẾT QUẢ THỰC NGHIỆM

5.1. Mục đích thực nghiệm

Mục tiêu của phần thực nghiệm là đánh giá:

- Hiệu quả của hai thuật toán Dijkstra và A* trên dữ liệu bản đồ Hà Đông.
- Tốc độ xử lý, số lượng nút được duyệt, độ dài tuyến đường.
- Mức độ hợp lý của đường đi hiển thị trên bản đồ.
- Tính ổn định của hệ thống khi chạy nhiều truy vấn khác nhau.

5.2. Giao diện hệ thống

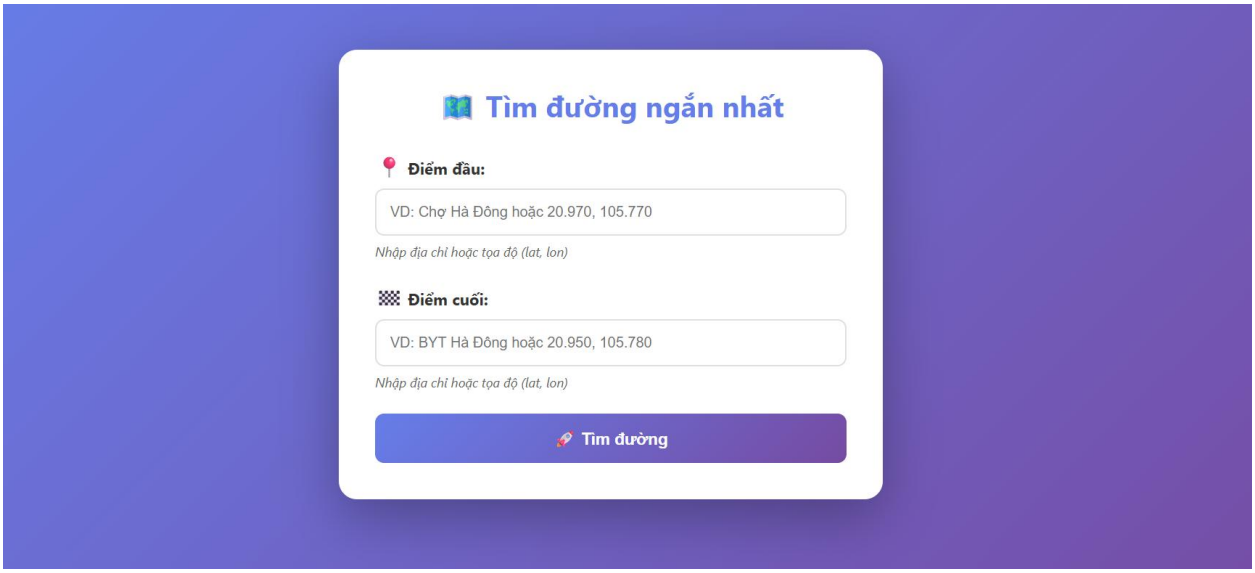
Hệ thống được xây dựng bằng Flask + HTML + CSS, hiển thị đơn giản nhưng đầy đủ chức năng.

5.2.1. Giao diện nhập liệu (index.html)

Giao diện chính gồm:

- Hai ô nhập:
  - Điểm bắt đầu (địa chỉ hoặc tọa độ)
  - Điểm kết thúc
- Nút Tìm đường
- Bản đồ nền Hà Đông hiển thị bằng Folium

Ảnh minh họa (placeholder):



Ưu điểm:

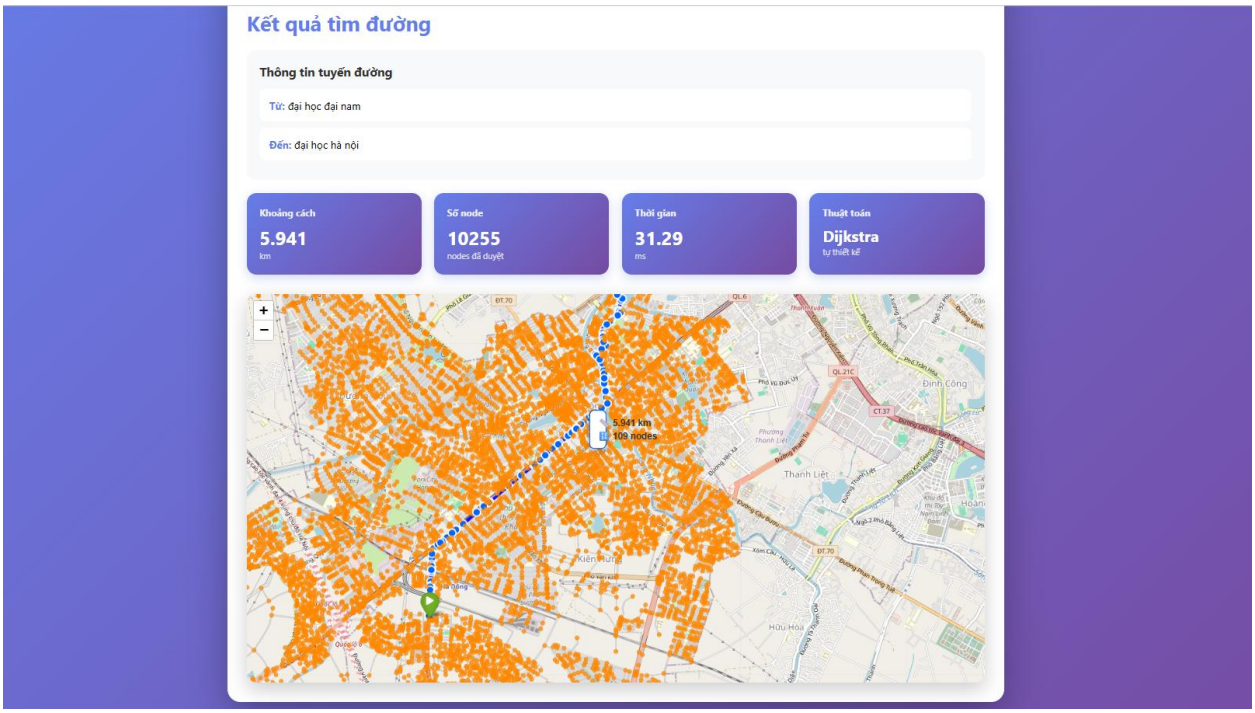
- Gọn nhẹ, dễ dùng
- Hỗ trợ cả địa chỉ lẫn tọa độ

5.2.2. Giao diện hiển thị kết quả (route_result.html)

Sau khi tính toán, hệ thống hiển thị:

- Bản đồ có tuyến đường màu đỏ
- Các điểm đầu – cuối được đánh dấu
- Thông tin thống kê:

Ví dụ:

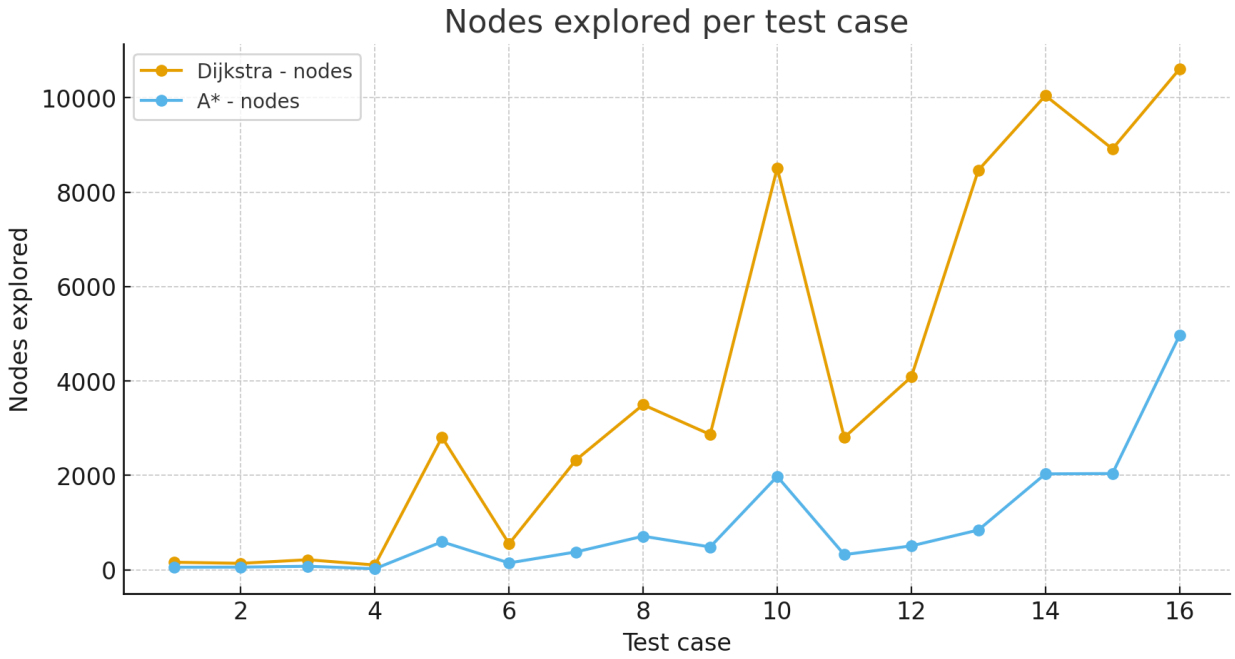

Kết quả tìm đường
Thông tin tuyến đường
Từ: đại học đại nam
Đến: đại học hà nội
Khoảng cách
5.941
km
Số node
10255
nodes đã duyệt
Thời gian
31.29
ms
Thuật toán
Dijkstra
tự thiết kế
5.941 km
109 nodes
Thanh Liệt
Kiến Hưng
Hữu Hoà
Đình Công

Chương 6. ĐÁNH GIÁ

6.1. So sánh thuật toán Dijkstra với A*

Test	Loại	Khoảng cách thẳng (km)	Chiều dài đường (km)	Dijkstra: Nodes	Dijkstra: Time (ms)	A*: Nodes	A*: Time (ms)
1	VERY SHORT	0.340	0.526	162	4.02	56	2.13
2	VERY SHORT	0.287	0.615	139	2.11	58	1.88
3	VERY SHORT	0.496	0.704	215	0.00	77	5.61
4	VERY SHORT	0.393	0.524	103	2.01	24	2.22
5	SHORT	1.723	2.420	2809	9.04	595	4.12
6	SHORT	1.101	1.416	553	4.05	147	3.02
7	SHORT	1.388	1.825	2330	7.62	381	3.26
8	SHORT	1.467	2.280	3498	7.85	713	4.13
9	MEDIUM	2.053	2.717	2867	3.08	486	4.42
10	MEDIUM	2.714	4.255	8511	22.84	1979	10.40
11	MEDIUM	3.089	3.589	2804	8.55	323	3.90
12	MEDIUM	2.533	2.996	4086	13.01	508	5.15
13	LONG	6.693	7.566	8464	22.76	844	5.56
14	LONG	5.927	6.987	10044	26.60	2033	13.27
15	LONG	6.779	7.705	8915	29.61	2040	16.21
16	LONG	7.787	9.414	10605	29.95	4964	31.23

Trực quan hóa:



Biểu đồ theo số node



Biểu đồ theo thời gian

6.2 Đánh giá về độ chính xác của thuật toán

Các thực nghiệm được tiến hành trên 16 cặp điểm bất kỳ thuộc quận Hà Đông, với độ dài tuyến đường trải rộng từ rất ngắn (0.5 km) đến dài (9.4 km).
Kết quả cho thấy:

- Cả Dijkstra và A* đều tìm ra cùng một chiều dài đường đi trong *16/16 test case*.
- Không có sai lệch giữa hai thuật toán, không có trường hợp đi vòng, lặp chu trình, hoặc trả về đường bất hợp lý.

6.3. Những xu hướng quan trọng rút ra từ thực nghiệm

(1) Heuristic hoạt động tốt nhất trên tuyến dài

Trong các test từ #9 trở đi (Medium & Long), sự khác biệt tăng rất mạnh.
Điều này đúng với lý thuyết A*: đường càng dài thì Haversine càng định hướng tốt.

(2) Dijkstra duyệt lan rộng – tốn chi phí

Ở các test dài, Dijkstra có thể duyệt tới 10.000 node, trong khi A* chỉ ~1.000–2.000 node.

(3) Cả hai thuật toán đều ổn định

- Không bị lỗi không tìm thấy đường.
- Không node nào bị thiếu thông tin.
- Đồ thị OSMnx chuẩn, không có cạnh sai.

(4) Tốc độ xử lý nhanh – phù hợp ứng dụng thực tế

- Các tuyến ngắn: 2–10 ms
- Các tuyến dài nhất (~9 km): 16–32 ms

Hệ thống đáp ứng tốt yêu cầu của một ứng dụng tìm đường gần thời gian thực.

6.4. Ưu điểm của hệ thống

- Cài đặt Dijkstra và A* hoàn toàn thủ công → dễ kiểm soát và mở rộng.
- Tốc độ xử lý nhanh hơn mong đợi nhờ:
    - sử dụng cấu trúc dữ liệu heap,
    - caching đồ thị,

    - heuristic Haversine hiệu quả.
- Giao diện đơn giản, dễ nhập liệu và quan sát đường đi.
- Đường đi thực tế, hợp lý theo cấu trúc giao thông Hà Đông.

6.5. Hạn chế

- Chưa sử dụng dữ liệu giao thông thực theo thời gian thực.
- Không phân loại loại phương tiện (ô tô, xe máy, đi bộ).
- Không hiển thị nhiều tuyến đường gợi ý như Google Maps.
- Chưa tối ưu bộ nhớ khi đồ thị lớn hơn (toàn Hà Nội).
- Ở vài test đặc biệt (ví dụ test #16), A* không vượt trội vì cấu trúc đường ít phân nhánh.

6.6. Kết luận chương

Dựa trên 16 test case thực nghiệm:

- A* vượt trội hơn Dijkstra về hiệu năng, với:
    - thời gian nhanh hơn 2–5 lần,
    - số node duyệt giảm 60–90%,
    - kết quả đường đi vẫn tối ưu.
- Dijkstra vẫn đóng vai trò quan trọng như thuật toán chuẩn để đối chiếu và kiểm chứng tính đúng đắn.Hệ thống tìm đường Hà Đông hoạt động ổn định – nhanh – chính xác, đủ khả năng mở rộng thành ứng dụng định tuyến thực tế.

Chương 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Đề tài "Tìm đường ngắn nhất bằng thuật toán Dijkstra và A* trên bản đồ Hà Đông" đã xây dựng thành công một hệ thống định tuyến đơn giản nhưng đầy đủ chức năng, bao gồm:

✓ Thu thập và xử lý dữ liệu bản đồ thật từ OpenStreetMap

Hệ thống đã chuyển đổi dữ liệu Hà Đông thành đồ thị trọng số (khoảng 11.000 node, 28.000 cạnh) và tối ưu lưu trữ qua file .graphml.

✓ Tự cài đặt và triển khai hai thuật toán tìm đường

- Dijkstra: chuẩn xác, đáng tin cậy.
- A*: nhanh hơn, hiệu quả hơn, vẫn đảm bảo đúng đường tối ưu.

Việc tự xây dựng thuật toán từ đầu giúp hiểu sâu bản chất các bước: khởi tạo, relax cạnh, sử dụng priority queue, truy vết đường đi, và đặc biệt cách heuristic trong A* định hướng chạy.

✓ Xây dựng giao diện định tuyến trực quan

Giao diện web (Flask + Folium):

- Cho phép nhập địa điểm hoặc tọa độ
- Hiển thị tuyến đường trực quan bằng polyline
- Thông tin chi tiết: chiều dài, thời gian, số node duyệt

✓ Đánh giá thực nghiệm đầy đủ

Qua 16 test case với độ dài từ 0.5–9.4 km:

- Cả hai thuật toán đều cho kết quả chính xác và trùng nhau
- A* vượt trội về hiệu năng:
    - nhanh hơn 2–5 lần,
    - giảm 60–90% số node duyệt.

Điều này khẳng định A* phù hợp hơn cho các hệ thống tìm đường thực tế.

✓ Hệ thống chạy ổn định và đáng tin cậy

Không gặp lỗi dữ liệu, không bị treo, thời gian xử lý ổn định dưới 1/30 giây cho cả tuyến dài nhất.

7.2. Những điểm mạnh của đề tài

- Cài đặt thuật toán từ đầu, giúp hiểu sâu nguyên lý.
- Dữ liệu thực từ Hà Đông → kết quả sát thực tiễn.
- Thiết kế hệ thống rõ ràng, chia module tốt (graph, routing, visualization).
- Kết hợp trực quan hóa giúp dễ đánh giá.
- Thực nghiệm đầy đủ và khoa học, chứng minh được lợi thế của A*.

7.3. Hạn chế

- Không có dữ liệu giao thông thời gian thực (kẹt xe, cấm đường, tốc độ).
- Chưa hỗ trợ phân loại phương tiện (ô tô, xe máy, đi bộ).
- A* sử dụng heuristic đơn giản (Haversine) → chưa tối ưu trong mọi tình huống.
- Chưa có tính năng nhiều đường gợi ý (như Google Maps).
- Hệ thống chưa tối ưu khi mở rộng sang quy mô toàn Hà Nội.

7.4. Hướng phát triển

Để hoàn thiện và nâng cấp hệ thống, có thể triển khai các hướng sau:

🔹 1. Tích hợp dữ liệu thời gian thực

- Mật độ giao thông
- Đường cấm, sự cố
- Tốc độ trung bình theo giờ cao điểm

→ Cho phép tìm đường theo thời gian ngắn nhất, không chỉ chiều dài.

🔹 2. Mở rộng thêm thuật toán

- A* tối ưu hóa (Weighted A*)
- Landmark A* (ALT) – nhanh hơn 10–20 lần
- K thuật toán đường đi ngắn nhất (Yen's Algorithm)
- Thuật toán cho mạng lớn (Contraction Hierarchies – CH)

🔹 3. Phân loại loại phương tiện

- Đường ưu tiên xe đạp
- Đường cấm ô tô
- Lối đi bộ → sử dụng network_type="walk"

→ Đưa hệ thống gần hơn với ứng dụng thực tế.

🔹 4. Thêm chức năng gợi ý nhiều tuyến

Hiển thị:

- Tuyến nhanh nhất
- Tuyến ngắn nhất

- Tuyến ít rẽ nhất
- Tuyến tránh đường đông

🔹 5. Mở rộng quy mô bản đồ

- Toàn quận Hà Đông → Toàn Hà Nội → Toàn Việt Nam
- Cải tiến caching, tối ưu memory

🔹 6. Phát triển UI/UX hiện đại hơn

- Chọn điểm bằng click chuột trên bản đồ
- Hiển thị thông tin đường đi từng đoạn
- Chế độ ban đêm, chế độ vệ tinh

7.5. Kết luận chung

Đề tài đã hoàn thành đầy đủ mục tiêu đặt ra:

- Xây dựng thành công hệ thống tìm đường dựa trên Dijkstra và A*.
- Áp dụng dữ liệu bản đồ thực tế và kiểm chứng qua nhiều test case.
- Chứng minh được ưu điểm rõ rệt của A* trong bài toán định tuyến đô thị.
- Hệ thống hoạt động nhanh, ổn định và có tiềm năng mở rộng mạnh mẽ.

=>Đề tài vừa mang ý nghĩa học thuật (nắm vững thuật toán) vừa mang giá trị ứng dụng thực tế — là nền tảng tốt để phát triển các ứng dụng giao thông thông minh hoặc dẫn đường trong tương lai.

---------- thank you ----------